

Lab Instructions 8.2 – Unnamed Semaphore.

Semaphores are used to synchronize processes and threads. Semaphores are combined with Message Queues and Shared Memory in the underlying IPCs in Unix systems. There are two types of semaphores, traditional System-V semaphores and POSIX semaphores. In this lesson, POSIX semaphores will be introduced.

With other synchronous solutions, the waiting thread still consumes CPU to run lock status checks – called “busy waiting”, which leads to system overhead. With a semaphore, the thread that fails to request this resource will freeze and will be woken up by the operating system when the resource becomes available again. During that time, the thread does not consume CPU.

There are two types of POSIX semaphores - named and unnamed. This LAB 8.2 will talk about unnamed semaphores, used internally between threads OR parent – child process.

Target	Related theory	Resources
Unnamed semaphore	Ch6.6.1 Semaphore Usage	https://github.com/Trantin84/LAB_IntroOS (example source code). Use Ubuntu 16/18 image
Operations on semaphores		
Apply to PC problem		

Ask students: Understand the theory of semaphore. Apply to synchronization problems.

Student reviews: Questions and answers about theoretical issues. Practical skills. Exercise.

Submission requirements: .c source code files and .out executable files of the Practice Session Requirements and Final Tutorial Exercises within the time allowed by the instructor.

Preferences

[1] Abraham Silberschatz, Peter B. Galvin, Greg Gagne, [2018], Operating System Concepts, 10th edition, John Wiley & Sons, New Jersey.

Programming Problems of Chapter 6.

[2] Linux manual page, [2021], on `sem_init(3)`,

Access https://man7.org/linux/man-pages/man3/sem_init.3.html

Requirements 1: Synchronize the Producer – Consumer problem using semaphore.

Instruct: Download file LAB_IntroOS/LAB_8/task2_1_PC_sem.c, compile and execute.

- Line 6 is the necessary library.
- Line 34 is the sem declaration.
- Line 38 initializes the semaphore with the value 1 (third argument). In this case, the equivalent semaphore is called a binary semaphore, equivalent to a mutex lock, and the purpose is to control the risk area.

For the second argument, the value 0 is used here, which should be shared between threads and declared globally on line 34. If this value is different from 0, the semaphore is understood to be used between parent processes. – child and semaphore then need to be stored in: shared memory (See LAB 5).

- Line 58 is to cancel the semaphore after use.
- Line 72 (corresponding to line 89) is the signal to wait for entering the dangerous area. If the sem value is > 0 , the process will enter; Otherwise ($\text{sem} \leq 0$) the thread will freeze.
- Line 76 is the Producer's risk area, while line 93 is the Consumer's risk area.
- Line 77 (corresponding to line 94) signals the completion of the dangerous area (then the sem value increases by 1 and will wake up the frozen process).

6	<code>#include <semaphore.h></code>
34	<code>sem_t mutex_sem;</code>
38	<code>sem_init(& mutex_sem, 0, 1)</code>
58	<code>sem_destroy(& mutex_sem)</code>
72(89)	<code>sem_wait(& mutex_sem)</code>
76(93)	<code>count++ / count--</code>
77(94)	<code>sem_post(& mutex_sem)</code>

Requirement 2: Solve the problem “Farmers cross the Vermont bridge” (see appendix) using Counting semaphore. For simplicity, the first version only considers bridge load = n (maximum n people on the bridge) with n passed in through the call.

Instruct: Download file LAB_IntroOS/LAB_8/task2_2_Bridge.c, compile and execute.

- Line 14 is the hypothetical number of farmers.
- Lines 18 to 21 are the behavior of a farmer: coming to the bridge, getting on the bridge, taking 3 to 7 seconds to cross, leaving the bridge.
- Line 29 is to create the farmers and line 30 assumes that the next farmer comes in 0 to 2 seconds.

14	#define MAX_FARMER 10
18	printf("\n%d Arriving bridge ...", v);
19	printf("\n%d Enter bridge ...", v);
20	sleep(rand() % 5 + 3);
21	printf("\n%d Leaving bridge ...", v);
29	pthread_create()
30	sleep(rand() % 3);

After implementation, we see that the number of farmers on the bridge is arbitrary. Suppose we need to control the bridge load, declare a counting semaphore and control as follows:

n is the load to be controlled.	sem_init(& mutex_sem, 0, n)
Before “Enter”	sem_wait(& mutex_sem)
After “Leaving”	sem_post(& mutex_sem)

Please recompile and run the edited program, discuss whether the bridge load is controlled or not? And why?

Programming exercises.

1. In Requirement 1 (LAB_IntroOS/LAB_8/task2_1_PC_sem.c) we have synchronously protected the count variable. However, there are still two places that cause busy waiting and consume system resources. That is:

69	while (count == BUFFER_SIZE); //do nothing
eig	while (count == 0); //do nothing

hty sev en	
------------------	--

Let's create 2 semaphores full and empty, in which full replaces line 69 to control when the buffer is full and freezes the Producer process while sem empty replaces line 87 which will freeze the Consumer process when the buffer is empty. What is the initial value per sem?

Reference: Slide Ch7. 4-6 and book [1] p. 290.

2. See the problem Estimating PI (LAB_IntroOS/LAB_7/task1_2_PI.c)

Let's do the synchronization using semaphore, compare the accuracy of the PI number afterwards. At the same time, measure the running time and indicate how much % the running time has increased after synchronization. Can the program be improved to minimize this % runtime cost?

3. Consider the problem "Farmer crossing the Vermont bridge", (let's call this exercise version 2), we do not control the load, but assume that the farmer comes from one of two directions: North and South, and there cannot be two people going in the same direction on the bridge. Let's design a semaphore to solve this problem.
4. Consider the problem "Farmers crossing the Vermont bridge", if we control both the load and the direction of travel, how should we solve it?

Appendix

Exercise 8.30 A single-lane bridge connects two Vermont villages, North and South. Farmers in the two villages use this bridge to deliver their products to the neighboring town. The bridge could become congested if a farmer from the north and south boarded the bridge at the same time, at the same time. (Vermont farmers are stubborn so they won't back down). Using semaphores and/or mutex locks, design an algorithm in pseudocode to prevent deadlocks. Initially, don't worry about resource exhaustion (because many farmers from one direction keep coming to the bridge and the other side has to wait indefinitely).